A Database Management System Project Report on

LIBRARY MANAGEMENT SYSTEM

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Submitted By:

Avinash MK - 2501010040
Shyam Nath Patro - 2501010130
Arijit Deb - 2501010037
Shaswat Dwivedi - 2501010090
Priyanshu Singh - 2501010017

Under the Guidance of:

Rishav Upadhyay

PW Institute of Innovation, Medhavi Skills University
School of Technology
Semester: 2nd | Academic Year: 2026

CERTIFICATE

This is to certify that the Database Management System project report entitled "Library Management System" is a bonafide record of the work carried out by the students under my supervision and guidance, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Engineering from PW Institute of Innovation, Medhavi Skills University.

The work embodied in this report is original and has not been submitted to any other University or Institution for the award of any degree or diploma.

_____

Rishav Upadhyay
Project Guide

_____

The Students
PW Institute of Innovation

ACKNOWLEDGEMENT

We would like to express our profound gratitude to all those who have been instrumental in the successful completion of this project. First and foremost, we extend our sincere thanks to our esteemed institution, PW Institute of Innovation, and the Department of Computer Science and Engineering for providing us with the necessary infrastructure and environment to undertake this project.

We are deeply indebted to our project guide, Rishav Upadhyay, for their invaluable guidance, continuous encouragement, and constructive feedback throughout the course of this project. Their profound knowledge and keen insights have been a constant source of inspiration.

Finally, we would like to thank our family and friends for their unwavering support and motivation during the challenging phases of our academic journey.

- Avinash MK, Shyam Nath Patro, Arijit Deb, Shaswat Dwivedi, Priyanshu Singh

ABSTRACT

The Library Management System is a comprehensive, database-driven web application meticulously designed to automate and streamline the cataloging, borrowing, and fine management operations of a modern library. Traditional paper-based systems are highly susceptible to human error, data redundancy, and slow retrieval. This project addresses these challenges by providing a robust digital infrastructure built upon a normalized MySQL relational database.

The system manages eight core entities: Publisher, Category, Author, Staff, Member, Book, Borrow Transaction, and Fine. The database enforces strict referential integrity through primary and foreign key constraints, ENUM restrictions, and CHECK constraints. The backend is powered by a Node.js Express server exposing a RESTful API, while the frontend is a responsive dark-themed Single Page Application built with HTML5, CSS3, and vanilla JavaScript.

Advanced SQL features including multi-table JOINs, nested subqueries, aggregate functions, GROUP BY with HAVING clauses, and predefined Views are implemented to fulfill complex reporting requirements. The system successfully demonstrates all mandatory DBMS capstone requirements: 3NF normalization, 151+ sample records, and full functional demonstration through a live web interface.

TABLE OF CONTENTS

1. INTRODUCTION

Libraries are among the oldest and most critical institutions of human civilization, serving as repositories of knowledge, culture, and information. With the exponential growth in published literature and digital resources, managing a library's operations manually has become increasingly inefficient and error-prone.

Traditional library management systems reliant on paper records or disconnected spreadsheets suffer from critical drawbacks: data redundancy, inability to track borrowing history, difficulty enforcing fine policies, and slow retrieval of book availability. These inefficiencies directly impact the library's primary mission of providing accessible, organized knowledge.

To overcome these challenges, the Library Management System was conceptualized and developed as part of the DBMS Capstone Project. It is a full-stack, database-driven web application serving as a centralized hub for all library operations — from cataloging books and managing members to processing borrow transactions and calculating fines.

This project emphasizes fundamental and advanced concepts of Database Management Systems. The core of the application is a deeply analyzed, fully normalized relational database managed by MySQL 8.0. By implementing strict constraints, primary and foreign key relationships, and optimized query structures, the system guarantees high performance, data integrity, and scalability.

2. PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Many libraries still rely on manual or partially digitized processes that lead to the following critical issues:

- Data Inconsistency: Book records, member information, and borrow logs stored in disconnected files lead to conflicting data and stale records.
- Redundancy: Member details and book information are often duplicated across multiple logs, wasting storage space and increasing the likelihood of update anomalies.
- No Real-time Availability: Librarians cannot instantly determine whether a specific book is available without physically checking the shelf.
- Poor Fine Management: Calculating and tracking overdue fines manually is time-consuming and prone to errors, leading to revenue loss.
- Weak Reporting: Generating borrowing history, member activity reports, or overdue lists is cumbersome without a structured database.

2.2 Objectives

The primary goal of this Capstone Project is to design, implement, and demonstrate a comprehensive relational database solution. Specific objectives include:

- Relational Modeling: To architect a structurally sound ER model encompassing Members, Books, Authors, Publishers, Staff, and Transactions.
- Data Normalization: To normalize the database schema up to the Third Normal Form (3NF) to eliminate insertion, update, and deletion anomalies.
- Interactive UI Development: To build a responsive, dark-themed frontend dashboard for seamless library operations.
- Backend API Integration: To develop a Node.js Express server that processes API requests and executes SQL queries dynamically.
- Advanced SQL Utilization: To implement complex SQL concepts such as multi-table Joins, nested queries, Views, and Aggregate functions.
- Fine Management: To automatically calculate and track overdue fines at Rs. 10 per day through transactional SQL operations.

3. SYSTEM ANALYSIS

3.1 Functional Requirements

- Dashboard Analytics: Dynamically aggregate and display total books, available copies, active members, active borrows, overdue count, and pending fine amounts.
- Book Catalog Management: Search and view all books with full details including authors (via GROUP_CONCAT), publisher, category, availability, and shelf location.
- Member Management: View all registered members with their membership type, status, borrow history count, and expiry date.
- Borrow Operations: Issue books to members with automatic due date calculation (14-day loan period) and update available copies atomically via transactions.
- Return Processing: Process book returns, automatically update availability, and calculate overdue fines at Rs. 10 per day when applicable.
- Fine Tracking: View all fines with paid/pending status, member details, and fine amounts.
- Overdue Reporting: Generate real-time overdue reports using predefined SQL Views.

3.2 Non-Functional Requirements

- Performance: Backend API processes CRUD requests efficiently using MySQL connection pooling. Frontend updates asynchronously via Fetch API without full page reloads.
- Data Integrity: Database enforces ACID properties. Borrow and return operations use MySQL transactions with ROLLBACK on failure, ensuring no partial updates.
- Referential Integrity: All foreign key relationships enforced at the database level, preventing orphaned records.
- Security: Parameterized queries used throughout the Node.js backend to prevent SQL injection attacks.
- Usability: High-contrast dark theme with CSS variables, sidebar navigation, and toast notifications for user feedback.

4. ER DIAGRAM & RELATIONAL SCHEMA

4.1 Entity-Relationship Model

The ER model identifies eight primary entities and their relationships in the Library Management System:

Entity	Description	Key Relationships
Publisher	Book publishers with contact details	1:N with Book
Category	Book genre/classification	1:N with Book
Author	Book authors with nationality	M:N with Book (via Book_Author)
Staff	Library staff who process transactions	1:N with Borrow_Transaction
Member	Registered library members	1:N with Borrow_Transaction
Book	Central entity — all book details	M:N with Author, FK to Publisher/Category
Borrow_Transaction	Borrow/return event records	FK to Member, Book, Staff
Fine	Overdue fine records	1:1 with Borrow_Transaction

4.2 Relational Schema

Core Entity Tables:

```
Publisher(publisher_id PK, name, address, phone UNIQUE, email UNIQUE,
established_year)
Category(category_id PK, name UNIQUE, description)
Author(author_id PK, first_name, last_name, nationality, birth_year)
Staff(staff_id PK, first_name, last_name, role ENUM, email UNIQUE, phone, hire_date)
Member(member_id PK, first_name, last_name, email UNIQUE, phone, address,
       membership_type ENUM, join_date, expiry_date, is_active)
Book(book_id PK, isbn UNIQUE, title, publisher_id FK, category_id FK,
     publication_year, edition, total_copies, available_copies, shelf_location)
```

Junction & Transaction Tables:

```
Book_Author(book_id FK, author_id FK)  -- Composite PK -- resolves M:N
Borrow_Transaction(transaction_id PK, member_id FK, book_id FK, staff_id FK,
                   borrow_date, due_date, return_date, status ENUM)
Fine(fine_id PK, transaction_id FK UNIQUE, amount, reason, paid, paid_date)
```

5. NORMALIZATION

Normalization is a systematic approach to decomposing tables to eliminate data redundancy and undesirable characteristics like Insertion, Update, and Deletion anomalies. The database strictly adheres to normalization principles up to the Third Normal Form (3NF).

5.1 First Normal Form (1NF)

A relation is in 1NF if every attribute is atomic (single-valued) with no repeating groups.

Application in Project: All member and book names are split into first_name and last_name columns. Phone and email are stored in distinct columns. Multiple authors for a single book are NOT stored as a comma-separated list in the Book table — instead, the many-to-many relationship is resolved externally through the Book_Author junction table. Every table has a well-defined primary key.

5.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and all non-key attributes are fully functionally dependent on the entire primary key. This is relevant for tables with composite primary keys.

Application in Project: Core tables (Book, Member, Author) use single-column surrogate keys (AUTO_INCREMENT), inherently satisfying 2NF. The junction table Book_Author uses a composite key (book_id, author_id) and contains only the key mapping — no additional attributes — ensuring no partial dependencies exist.

5.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and there are no transitive dependencies — every non-key attribute depends only on the primary key, not on another non-key attribute.

Application in Project:

- Publisher details (name, address, phone) are stored in the Publisher table, not inside Book. This eliminates the transitive dependency: book_id -> publisher_id -> publisher_name.
- Category details are stored in the Category table, not inside Book.

- Member details are not duplicated in Borrow_Transaction. The transaction only stores member_id as a foreign key.
- Fine attributes (amount, reason, paid) depend solely on fine_id, not on the book or member associated with the transaction.

Functional Dependencies (key examples):

Table	Functional Dependency	NF Compliance
Book	book_id -> title, isbn, publisher_id, category_id, total_copies	3NF
Member	member_id -> first_name, last_name, email, membership_type	3NF
Borrow_Transaction	transaction_id -> member_id, book_id, borrow_date, status	3NF
Fine	fine_id -> transaction_id, amount, paid	3NF
Book_Author	(book_id, author_id) -> (no additional attributes)	3NF

6. DATABASE DESIGN & DATA DICTIONARY

6.1 Table: Member

Attribute	Data Type	Constraints	Description
member_id	INT	PK, AUTO_INCREMENT	Unique member identifier
first_name	VARCHAR(50)	NOT NULL	Member's given name
last_name	VARCHAR(50)	NOT NULL	Member's family name
email	VARCHAR(100)	UNIQUE, NOT NULL	Email address
phone	VARCHAR(15)	—	Contact number
membership_type	ENUM	NOT NULL	Student / Faculty / Public
join_date	DATE	NOT NULL	Membership start date
expiry_date	DATE	NOT NULL	Membership expiry date
is_active	BOOLEAN	DEFAULT TRUE	Active status flag

6.2 Table: Book

Attribute	Data Type	Constraints	Description
book_id	INT	PK, AUTO_INCREMENT	Unique book identifier
isbn	VARCHAR(20)	UNIQUE, NOT NULL	International Standard Book Number
title	VARCHAR(200)	NOT NULL	Book title
publisher_id	INT	FK -> Publisher	Publisher reference
category_id	INT	FK -> Category	Category reference
edition	INT	DEFAULT 1	Edition number
total_copies	INT	NOT NULL	Total copies in library
available_copies	INT	CHECK >= 0	Currently available copies
shelf_location	VARCHAR(20)	—	Physical shelf code

6.3 Table: Borrow_Transaction

Attribute	Data Type	Constraints	Description
transaction_id	INT	PK, AUTO_INCREMENT	Unique transaction identifier
member_id	INT	FK -> Member	Borrowing member
book_id	INT	FK -> Book	Borrowed book
staff_id	INT	FK -> Staff	Staff who processed it
borrow_date	DATE	NOT NULL	Date of borrowing
due_date	DATE	NOT NULL	Return deadline
return_date	DATE	DEFAULT NULL	Actual return date
status	ENUM	DEFAULT Borrowed	Borrowed / Returned / Overdue

6.4 Table: Fine

Attribute	Data Type	Constraints	Description
fine_id	INT	PK, AUTO_INCREMENT	Unique fine identifier
transaction_id	INT	FK UNIQUE -> Transaction	Associated transaction
amount	DECIMAL(8,2)	CHECK > 0, NOT NULL	Fine amount in INR
reason	VARCHAR(200)	—	Reason for fine
paid	BOOLEAN	DEFAULT FALSE	Payment status
paid_date	DATE	DEFAULT NULL	Date of payment

7. SQL IMPLEMENTATION

7.1 Creating Core Tables

```
CREATE TABLE Member (
    member_id       INT AUTO_INCREMENT PRIMARY KEY,
    first_name      VARCHAR(50) NOT NULL,
    last_name       VARCHAR(50) NOT NULL,
    email           VARCHAR(100) UNIQUE NOT NULL,
    membership_type ENUM('Student','Faculty','Public') NOT NULL,
    join_date       DATE NOT NULL,
    expiry_date     DATE NOT NULL,
    is_active       BOOLEAN DEFAULT TRUE
);
```

```
CREATE TABLE Book (
    book_id         INT AUTO_INCREMENT PRIMARY KEY,
    isbn            VARCHAR(20) UNIQUE NOT NULL,
    title           VARCHAR(200) NOT NULL,
    publisher_id    INT NOT NULL,
    category_id     INT NOT NULL,
    total_copies    INT NOT NULL DEFAULT 1,
    available_copies INT NOT NULL DEFAULT 1,
    shelf_location  VARCHAR(20),
    FOREIGN KEY (publisher_id) REFERENCES Publisher(publisher_id),
    FOREIGN KEY (category_id)  REFERENCES Category(category_id),
    CHECK (available_copies >= 0),
    CHECK (available_copies <= total_copies)
);
```

7.2 Junction Table with Composite Primary Key

```
CREATE TABLE Book_Author (
    book_id     INT NOT NULL,
    author_id   INT NOT NULL,
    PRIMARY KEY (book_id, author_id),
    FOREIGN KEY (book_id)   REFERENCES Book(book_id),
    FOREIGN KEY (author_id) REFERENCES Author(author_id)
);
```

7.3 Transactional Borrow Operation

The borrow operation uses MySQL transactions to ensure atomicity — if any step fails, the entire operation is rolled back:

```
BEGIN TRANSACTION;
  INSERT INTO Borrow_Transaction
    (member_id, book_id, staff_id, borrow_date, due_date, status)
    VALUES (?, ?, ?, ?, ?, 'Borrowed');
```

```
  UPDATE Book
    SET available_copies = available_copies - 1
    WHERE book_id = ? AND available_copies > 0;

  -- If affected rows = 0, book not available -> ROLLBACK
COMMIT;
```

8. VIEWS, JOINS & ADVANCED QUERIES

8.1 Views

```
-- View 1: Currently active borrows
CREATE VIEW vw_active_borrows AS
SELECT bt.transaction_id,
       CONCAT(m.first_name,' ',m.last_name) AS member_name,
       m.membership_type, b.title AS book_title,
       bt.borrow_date, bt.due_date, bt.status
FROM Borrow_Transaction bt
JOIN Member m ON bt.member_id = m.member_id
JOIN Book   b ON bt.book_id   = b.book_id
WHERE bt.status IN ('Borrowed','Overdue');
```

```
-- View 2: Overdue books with fine details
CREATE VIEW vw_overdue_with_fines AS
SELECT CONCAT(m.first_name,' ',m.last_name) AS member_name,
       m.email, b.title, bt.due_date,
       f.amount AS fine_amount, f.paid
FROM Borrow_Transaction bt
JOIN Member m ON bt.member_id = m.member_id
JOIN Book   b ON bt.book_id   = b.book_id
LEFT JOIN Fine f ON f.transaction_id = bt.transaction_id
WHERE bt.status = 'Overdue';
```

8.2 Multi-Table JOIN Query

```
-- Books with all authors and publisher (4-table JOIN)
SELECT b.title,
       GROUP_CONCAT(CONCAT(a.first_name,' ',a.last_name) SEPARATOR ', ') AS authors,
       p.name AS publisher, c.name AS category
FROM Book b
JOIN Book_Author ba ON b.book_id    = ba.book_id
JOIN Author      a  ON ba.author_id = a.author_id
JOIN Publisher   p  ON b.publisher_id = p.publisher_id
JOIN Category    c  ON b.category_id  = c.category_id
GROUP BY b.book_id ORDER BY b.title;
```

8.3 Nested Subquery

```
-- Members who borrowed more books than the average
SELECT CONCAT(m.first_name,' ',m.last_name) AS member,
       COUNT(bt.transaction_id) AS total_borrows
FROM Member m
JOIN Borrow_Transaction bt ON m.member_id = bt.member_id
GROUP BY m.member_id
HAVING COUNT(bt.transaction_id) > (
    SELECT AVG(borrow_count) FROM (
        SELECT COUNT(transaction_id) AS borrow_count
        FROM Borrow_Transaction GROUP BY member_id
```

```
    ) AS avg_table
);
```

8.4 GROUP BY with HAVING

```
-- Members who borrowed more than 2 books
SELECT CONCAT(m.first_name,' ',m.last_name) AS member,
       m.membership_type,
       COUNT(bt.transaction_id) AS total_borrows
FROM Member m
JOIN Borrow_Transaction bt ON m.member_id = bt.member_id
GROUP BY m.member_id, m.membership_type
HAVING COUNT(bt.transaction_id) > 2
ORDER BY total_borrows DESC;
```

9. FRONTEND, BACKEND & ARCHITECTURE

9.1 System Architecture

The system follows a three-tier architecture:

Layer	Technology	Responsibility
Frontend	HTML5, CSS3, JavaScript	UI rendering, user interaction, Fetch API calls
Backend	Node.js, Express.js	REST API, route handling, DB query execution
Database	MySQL 8.0 (Docker)	Data storage, constraints, views, transactions

9.2 Frontend Architecture

- HTML5: Single Page Application (SPA) structure with sidebar navigation and dynamic section toggling.
- CSS3: Dark theme using CSS custom properties (--bg: #0f1117, --accent: #6c63ff). Grid and Flexbox for responsive card layouts.
- JavaScript: Async Fetch API calls to the backend, DOM manipulation, toast notification system, and dynamic table rendering.

9.3 Backend Architecture

- Express.js: REST API server on port 3000 with CORS enabled. Routes organized by resource (books, members, borrows, fines).
- mysql2 Driver: Parameterized queries (Prepared Statements) for all database operations to prevent SQL injection.
- Transaction Management: Borrow and Return operations wrapped in MySQL transactions with automatic ROLLBACK on failure.
- Fine Calculation: Return endpoint automatically calculates days overdue and inserts Fine record when applicable (Rs. 10/day).

9.4 API Endpoints

Method	Endpoint	Description
GET	/api/stats	Dashboard aggregated statistics
GET	/api/books?search=	Book catalog with optional search
GET	/api/members	All members with borrow counts

Method	Endpoint	Description
GET	/api/borrows/active	Active borrows via vw_active_borrows view
GET	/api/borrows/overdue	Overdue report via vw_overdue_with_fines view
POST	/api/borrow	Issue book (transactional)
POST	/api/return/:id	Return book with auto-fine calculation
GET	/api/fines	All fines with member and book details

10. RESULTS & FEATURES

The implemented system successfully achieves all functional and technical objectives, providing library staff with a seamless web interface for complete library operations management.

Module	Feature	Status
Dashboard	Real-time stats: books, members, borrows, fines	Implemented
Book Catalog	Full catalog with search by title/author, availability badges	Implemented
Member Management	All members with type, status, and borrow history	Implemented
Active Borrows	Live borrow status from DB view	Implemented
Issue Book	Transactional issue with automatic due date (14 days)	Implemented
Return Book	Return processing with automatic fine calculation	Implemented
Fine Tracking	All fines with paid/pending status	Implemented
Overdue Report	Real-time overdue report from DB view	Implemented

10.1 Database Statistics

Table	Record Count	Purpose
Publisher	10	Book publishers
Category	8	Book categories
Author	15	Book authors
Staff	5	Library staff
Member	25	Registered members
Book	20	Book catalog
Book_Author	25	Book-Author mappings
Borrow_Transaction	35	Borrow/return records
Fine	8	Overdue fine records
TOTAL	151	Exceeds 100+ requirement

11. CONCLUSION & FUTURE SCOPE

11.1 Conclusion

The Library Management System capstone project successfully demonstrates end-to-end development of a robust database application. By meticulously transitioning from conceptual ER modeling to logical schema design, and enforcing normalization up to the Third Normal Form (3NF), the project ensures absolute data integrity and operational efficiency.

The separation of concerns between the responsive HTML/CSS/JS frontend, the Express.js API backend, and the highly relational MySQL database mimics professional software engineering workflows. The system completely eradicates the redundancies associated with manual library record-keeping, enabling staff to manage complex operations — from book issuance to fine collection — with speed and accuracy.

All mandatory technical requirements of the DBMS Capstone have been fulfilled: proper relational schema design, 3NF normalization, 151+ sample records, all five SQL query categories (JOINs, nested queries, aggregate functions, GROUP BY/HAVING, Views), and a functional frontend-backend integration.

11.2 Future Scope

- User Authentication: JWT-based role-based access control (Admin Librarian vs. Standard Staff vs. Member portal).
- Online Member Portal: Allow members to search books, view their borrow history, and pay fines online.
- Email Notifications: Automated email reminders for due dates and overdue alerts using NodeMailer.
- Book Reservation System: Allow members to reserve books that are currently checked out.
- Barcode/QR Integration: Scan ISBN barcodes for faster book issuing and return processing.
- Analytics Dashboard: Advanced reporting with charts showing borrowing trends, popular categories, and peak hours.

— END OF REPORT —